Agentic Robotics Benchmark - Team Brief

Oct 23, 2026 · @Pawel Irzyk

Purpose

This brief proposes four benchmark tasks for agentic development on our robotics stack, shares what I expect from them, and lists what I can provide as a test environment. The architecture - how the agent observes and acts, how attempts are run and graded - is yours to propose.

The idea: an agent that works on a live system - the running robot, sim and RViz - not only on code. Problems that can be solved by reading code or git history are out of scope.

Proposed tasks

Four tasks, each a step up in complexity and each testing a different capability. Details, difficulty levels and grading are open for your proposal.

1. Catch unreported failures - observe

- *Tests:* understanding what the robot actually did, from camera images plus robot state, while it runs - and calling a healthy run healthy.
- *Why:* actions report success even when the outcome is wrong. We have seen the robot pick an untracked wire with every action succeeding.
- *Sketch:* pick-and-place episodes where some go wrong (wrong object, placed outside the crate, dropped in transit), mixed with healthy ones.
- *Starts with:* a stream of episodes - cameras, robot state, flow state - without ground truth.
- *Done when:* every episode is called healthy or failed, with what went wrong and when.

2. Calibrate a camera - closed-loop procedure

- *Tests:* running a full procedure without help: plan robot poses, collect data, estimate the camera pose, judge when the result is good enough, prove it with picks.
- *Why:* calibration is the biggest commissioning cost of a new cell and still mostly manual. Sim gives the exact true camera pose to grade against.
- *Sketch:* one camera's mounting pose is off by a few cm / degrees from its config; the agent has to find the true pose.

- *Starts with:* a running cell where one camera's configured pose is wrong, and the symptom "picks sometimes miss".
- *Done when:* the corrected pose is in config, matches the true pose, and picks succeed.

3. Find a resource bottleneck - diagnose below ROS

- *Tests:* linking a robot symptom (dropped point clouds, controller jitter, timeouts) to a host cause (CPU, RAM, GPU, network, DDS), and fixing the cause rather than hiding the symptom.
- *Why:* resource exhaustion shows up as robot misbehaviour, not as a clear error. We have hit this many times (overloaded host, GPU shared by sim and perception, DDS buffers).
- *Sketch:* a working pick-and-place run with one pressure source added; the agent only gets the symptom.
- *Starts with:* a degraded pick-and-place run and a symptom report, e.g. "picks got slow and sometimes time out".
- *Done when:* the culprit is named, the cause is fixed, and the run is back to normal performance.

4. Investigate an open timing bug - open investigation

- *Tests:* forming hypotheses, designing experiments, reasoning about timing across components, and backing every claim with evidence.
- *Why:* timing bugs take most of our real debugging time. MoveIt's octomap shows phantom obstacles around the moving arm - real, visible in RViz, unsolved.
- *Sketch:* first with a known injected delay (has a correct answer), then the real bug (graded by measurement).
- *Starts with:* the symptom - phantom obstacles around the arm, visible in RViz, blocking motion planning.
- *Done when:* the root cause is proven by experiment, and the fix measurably removes the phantom voxels.

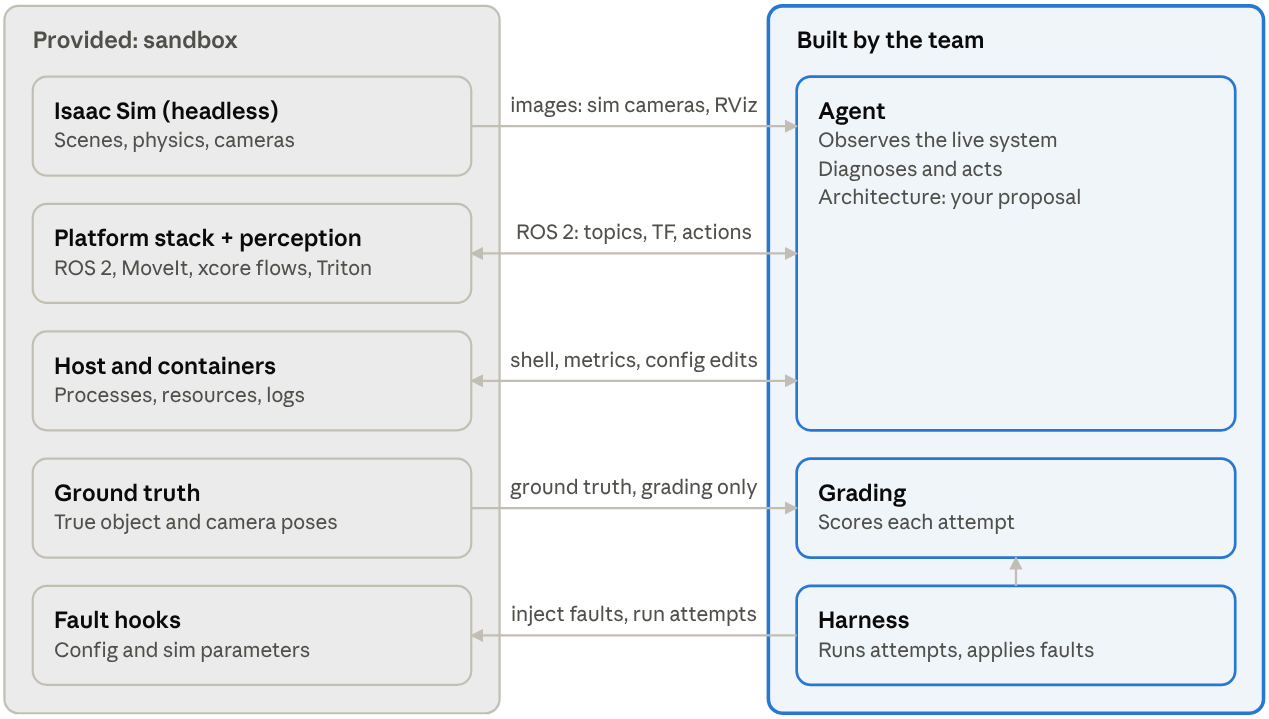
Expectations

- **Online and multimodal.** The agent works with the system while it runs - ROS graph, TF, logs, host metrics, and images from sim cameras and RViz read by a VLM. Not limited to ROS where the problem sits elsewhere.

- **Reproducible.** The same attempt gives a comparable result each time; faults are seeded, not random.
- **Objective grading.** Success is judged by what physically happened (sim ground truth, measurements), not by the agent's own report or by action results.
- **Healthy controls.** "Nothing is wrong" is a valid answer and must be tested - an agent that invents problems is worse than none.
- **Evidence over stories.** Conclusions are backed by data the agent collected.
- **Show what helps.** Compare against a plain coding agent as baseline, so we know what each added tool or modality (VLM included) actually contributes.

What I provide

The sandbox is provided; the agent, harness and grading are yours



sandbox vs team scope · interfaces between them

The agent sees the system only through these interfaces; ground truth reaches grading, never the agent.

What you get

- Docker image of our stack (built from the `ci` target) with pinned dependencies and RViz for headless renders.

- Scripted bring-up of headless Isaac Sim + Triton + stack for the pick-and-place scene, with a readiness check.
- Sim ground truth: object poses today, camera poses added for Task 2.
- Config and sim parameters your harness can change to inject faults.
- My time for stack questions and context on the real issues behind each task.

Running attempts, isolating them, injecting faults and grading are part of the harness you propose.

Existing assets to reuse

- Pick-and-place sim cell: Doosan e0509, Robotiq gripper, 4 static cameras, 7 wires, 2 crates.
- System tests that already run pick-and-place end to end in sim and grade it by object displacement.
- Sim depth-noise model and our calibration tools.

Known limits: depending on the available resources, the dev box can not run sim + stack + agent together reliably, so live-sim tasks need a dedicated GPU host; headless sim needs a few minutes of warm-up before results are trustworthy.

Open for your proposal

- Agent architecture: how it observes the live system (streams, images, RViz, host metrics) and how it acts.
- Harness: how attempts are run, faults injected, results collected and graded.
- Result format, metrics and pass criteria per task.
- Order of work, and any task you would change, merge or replace.
- What you need from the environment beyond the list above.

Next steps

1. Team reviews the tasks and comments here.
2. Clarification/sharpening on task definition if needed.
3. Team proposes the architecture and a first plan.
4. Once we agree on scope, I start on the test environment, so it is ready when you start building.